

TODO: Paper Title

Anonymous author

Anonymous affiliation

Abstract

Background. TODO: 1–2 sentences on the problem and why it matters.

Aims. TODO: the research questions or specific objectives.

Method. TODO: dataset, models, approaches compared.

Results. TODO: headline numbers, ordered by RQ.

Conclusions. TODO: what this means and the takeaway for practitioners.

2012 ACM Subject Classification Software and its engineering → Software notations and tools;
Computing methodologies → Natural language processing

Keywords and phrases issue classification, retrieval-augmented generation, fine-tuning, large language models, mining software repositories

Digital Object Identifier 10.4230/LIPICs...

1 Introduction

2 Related Work

Early IRC studies employed data mining techniques and ML classifiers [1, 18, 12]. For example, Antoniol et al. [1] leveraged a naive Bayes classifier to distinguish bugs from other issue reports. However, these methods showed low performance due to their limited semantic understanding of the issue report text [13].

To overcome this, recent methods have adopted transformer models due to their strong contextual understanding [6, 4, 24, 16, 17]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [16] fine-tuned a RoBERTa [21] model to classify issues into *Bugs*, *Features*, or *Questions*. Similarly, Trautsch et al. [24] trained a seBERT [25] model to classify issues into the same three categories. Although transformer-based approaches achieve strong results, they rely on large amounts of project-specific data to be effective, hindering their generalizability on projects with a small number of labeled issues [13, 16]. Additionally, they struggle to accurately classify minority classes with fewer training examples [17, 6].

These challenges have motivated the exploration of LLMs, which exhibit strong text classification capabilities thanks to extensive pretraining on massive corpora. [13, 3, 2, 5, 26, 8]. Studies show that LLMs perform well in zero-shot settings [8, 2]. SOTA LLM-based methods achieved the best performance by fine-tuning the models on labeled datasets [13, 3]. For example, Aracena et al. [3] instruction-tuned several models on the NLBSE '23 and '24 datasets.

3 Approach

3.1 Problem Formulation

Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or *question*. Only the issue's title and description text are used for classification with no other signals from the issue tracker. The defined label space is adopted directly from prior IRC studies [2, 3, 16, 18, 1].



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

42 3.2 Indexing and Retrieval

43 To prepare the issues for retrieval based on textual similarity, we first embed their textual
 44 content (i.e., *title + description*). We do not embed the issue labels. Instead, we retain
 45 them as metadata associated with each embedded issue. To generate the embeddings, we use
 46 the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [22]. We selected this
 47 model due to its lightweight architecture, fast performance, and effectiveness on issue report
 48 text [15]. The embeddings are then indexed using the Faiss vector database library [10],
 49 which provides efficient high-dimensional similarity retrieval. At the retrieval stage, the query
 50 issue is first embedded with the same model. Next, Faiss calculates the cosine similarity
 51 of indexed issue reports to the query, and returns the top-K nearest neighbors, ordered by
 52 descending similarity.

53 3.3 RAGTAG

54 RAGTAG is our proposed retrieval-augmented few-shot prompting approach for IRC. RAG-
 55 TAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors
 56 to the query issue, and present them as classified examples to the LLM in a few-shot prompt.
 57 This enables a more accurate classification, based on in-context learning [5]. The choice to
 58 present the top-K nearest neighbors as examples is based on two insights from prior research:
 59 First, retrieval-based example selection outperforms random selection [19]. Second, the most
 60 similar neighbors tend to share the same labels [9]; therefore, they are informative for the
 61 classification.

62 We create the few-shot prompt in three parts. First, the system message frames the
 63 classification task and the label space. Second, the top-K nearest neighbors are listed in
 64 retrieval order (descending similarity), each formatted as *title + description + label*. Finally,
 65 the query issue is appended to the end of the prompt, ensuring it receives high attention
 66 from the LLM [20]. $K = 0$ produces a zero-shot prompting baseline, similar to prior work [7].

67 We evaluate RAGTAG at top-K values in $\{0, 1, 3, 6, 9\}$. Details for the K value grid
 68 selection are provided in Section 5.1. Since the median prompt size at $K = 9$ in our dataset
 69 is $\sim 6,300$ tokens, the maximum sequence length is set to 8,192 tokens. Roughly 30% of the
 70 context quota is reserved for the query issue and 70% for the neighbors. If the neighbors
 71 exceed their overall quota, they are truncated proportionally based on their length (longer
 72 neighbors get more truncation). The query issue gets truncated if it exceeds its budget as
 73 well. Truncation happens on the right side of the description text, preserving all titles and
 74 labels (if any).

75 The LLM is instructed to generate the predicted label wrapped in XML tags (e.g.
 76 `<label>bug</label>`). The labels of the presented examples also follow the same format.
 77 This technique enables easy detection of the predicted label from the LLM’s generation, and
 78 reduces variance in LLM outputs caused by prompt format sensitivity [23]. We use a regular
 79 expression to extract the predicted label. Unparseable outputs are marked as *invalid*.

80 3.4 VOTAG

81 VOTAG is our proposed non-parametric retrieval IRC method. VOTAG uses the retrieval
 82 pipeline described in Section 3.2 to retrieve the top-K nearest neighbors of the query issue
 83 report, then predicts the final label with a similarity-weighted vote on their labels. This
 84 approach follows the standard K-nearest-neighbor classification setup [9], with neighbors

85 weighted by their similarity to the query [11].¹ Formally, for a query issue q with retrieved
 86 neighbors $\{(n_i, l_i, s_i)\}_{i=1}^K$, where n_i is the i -th nearest neighbor with label l_i and cosine
 87 similarity s_i to q , the label is predicted by:

$$88 \quad \hat{y} = \arg \max_{c \in \mathcal{L}} \sum_{i=1}^K s_i \cdot \mathbf{1}[l_i = c], \quad (1)$$

89 where $\mathcal{L}$ is the label set and $\mathbf{1}[\cdot]$ is the indicator function. VOTAG breaks ties by returning
 90 the label of the most similar neighbor in the tied classes.

91 VOTAG has three main roles: First, it serves as a fast and cost-efficient baseline that
 92 LLM-based approaches must clear to justify their computational cost. Second, it is used
 93 as an ablation to distinguish the contributions of the LLM and retrieval in the RAGTAG
 94 method. Third, VOTAG’s performance as a function of K informs the K value grid evaluated
 95 for RAGTAG.

96 3.5 Fine-Tuning

97 SOTA IRC approaches have achieved strong performance by supervised fine-tuning of
 98 LLMs [13, 3, 2], using Low-Rank Adaptation (LoRA) [14]. In these studies, the model is
 99 trained on labeled issue reports and predicts the labels for unseen test issues. Each training
 100 example is formatted as an instruction prompt which includes the issue’s title and description
 101 as input, and its label as output. At inference, the trained model receives the test issue in
 102 the same format, and predicts the empty label field. Regular expressions are used to extract
 103 the prediction from the output, and unparseable outputs are marked as *invalid*. We use this
 104 established methodology as our fine-tuning baseline.

105 For memory efficiency, each model is loaded in 4-bit quantization with Unsloth². The
 106 LoRA rank and alpha are set to 16, with a learning rate of 2×10^{-4} and the paged AdamW
 107 8-bit optimizer. Training runs for 3 epochs with a per-device batch size of 1 and gradient
 108 accumulation steps of 16. The max sequence length is set to 2,048 tokens, which covers
 109 94.9% of the issues in our dataset. Issues that exceed this budget get truncated from the
 110 right side of their description text to fit.

111 4 Experimental Setup

112 4.1 Dataset and Settings

113 We use the dataset introduced in Heo et al. [13] for our experiments. The dataset has a total
 114 of 6,600 issue reports gathered from eleven active open-source projects: `ansible/ansible`,
 115 `bitcoin/bitcoin`, `dart-lang/sdk`, `dotnet/roslyn`, `facebook/react`, `flutter/flutter`,
 116 `kubernetes/kubernetes`, `microsoft/TypeScript`, `microsoft/vscode`, `opencv/opencv`, and
 117 `tensorflow/tensorflow`. Each project has 600 issue reports with an equal distribution
 118 across the labels (*bug*, *feature*, *question*). To support their LLM fine-tuning method, Heo et
 119 al. divided the dataset into 50/50 train and test splits, stratified by project and label. This
 120 results in 300 train and 300 test issue reports per project.

121 We also adopt Heo et al.’s project-specific (PS) and project-agnostic (PA) configurations
 122 to examine how data scope affects IRC performance. In the PS setting, each project’s train

¹ Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

² <https://unsloth.ai>

and test data are used in isolation, resulting in eleven different train-test pairs. In the PA setting, the train splits of all projects are merged into a single training set of 3,300 issue reports. In both configurations, RAGTAG and VOTAG only use the training splits as their retrieval index to ensure fair comparison with the fine-tuning baseline.

4.2 Models

We use the Qwen2.5-Instruct model family [27] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance on general-purpose benchmarks. Using the same model at different parameter sizes isolates the effect of model scale from model architecture across LLM-based IRC approaches. Model temperature is set at 0.1 for deterministic classifications.

4.3 Evaluation Metrics

We report macro-averaged precision, recall, and F_1 over the three labels (**bug**, **feature**, **question**), along with overall accuracy. Per-class metrics are reported when the asymmetry across classes is itself the subject of analysis. Outputs that fail to parse a valid label (after the XML-tag extraction and regex fallback described in ??) are counted as incorrect, and an *invalid rate* is reported separately so any parsing-induced loss is auditable.

4.3.0.1 Aggregation convention (PA vs PS).

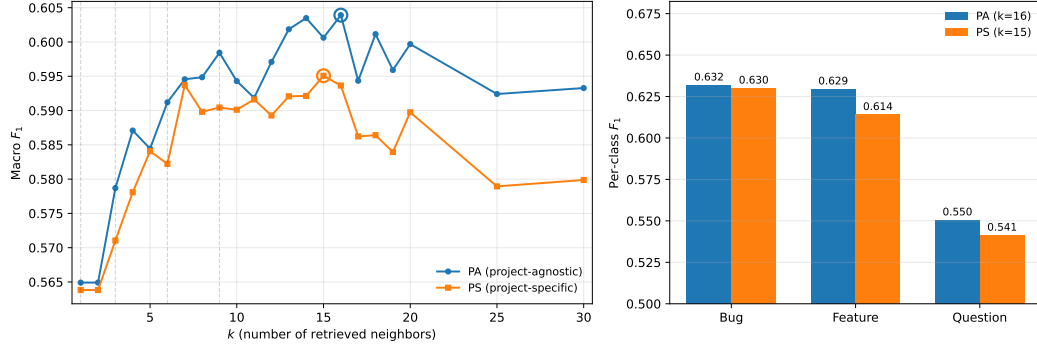
The PA setting produces a single set of predictions over the full 3,300-issue test set, and we compute macro F_1 directly over that set. The PS setting produces eleven sets of predictions, one per project, each over its own ~ 300 -issue test slice. Throughout the paper we report PS results using *pooled* aggregation: the eleven per-project prediction sets are concatenated into one 3,300-issue test set and macro F_1 is computed once on the pool. This treats each test issue as one observation in both settings and yields apples-to-apples comparisons between PA and PS. The alternative — computing macro F_1 separately within each project and averaging the eleven numbers — weights projects rather than issues; because macro F_1 is non-linear in the underlying confusion-matrix counts, the two conventions disagree, and on our data the per-project mean is consistently 0.002–0.011 macro F_1 below the pooled value with the gap largest for fine-tuning. We adopt pooling as the canonical reporting convention to remove this asymmetry; per-project breakdowns appear only where the project is the unit of analysis and are explicitly labelled as such.

4.4 Setup Hardware

5 Evaluations

5.1 RQ1) To what extent can similarity-weighted voting on the nearest neighbors (VOTAG) classify issue reports, and at what point does adding neighbors stop helping?

We evaluate VOTAG’s performance on both PA and PS settings at k values 1 to 30. The resulting macro F1 scores are shown in Figure 1. VOTAG’s best macro F1 is 0.595 ($k=15$) in PS and 0.604 ($k=16$) in PA. Both settings reach 99.8% (PS) and 98.5% (PA) of their respective peak by $k=7$. When adding neighbors beyond the peak, PS loses 0.015 and PA loses 0.011 by $k=30$.



■ **Figure 1 (left)** VOTAG macro F_1 as a function of k in both PS and PA settings. Circles mark peak performance. **(right)** Per-class F_1 at each setting’s best k ($k=15$ for PS, $k=16$ for PA).

PA consistently outperforms PS at every k . However, the performance gap is marginal: averaging 0.007 macro F_1 , with a maximum of 0.015 at $k=18$. This similar performance is due to the large overlap of the top- k neighbors in both settings. Specifically, PA and PS share 84–90% of the top- k neighbors across $k \in [3, 30]$. Therefore, even when using the full 3,300-issue retrieval index, most of the retrieved neighbors are from the same project. This means issue reports from the same project tend to be closer in the embedding space.

Question consistently has the weakest performance among the three labels at every k in both settings. At best k , PS per-class F_1 is 0.630 for bug, 0.614 for feature, and 0.541 for question. The corresponding PA values are 0.632, 0.629, and 0.550 (Figure 1). Additionally, questions are more frequently misclassified as bugs: 32% of question issues are predicted as bugs in both settings, while 18% (PS) and 17% (PA) are predicted as features. Since VOTAG predicts the label with the most accumulated similarity weight, this misclassification asymmetry indicates that question issues are more similar to bugs than to features in the embedding space.

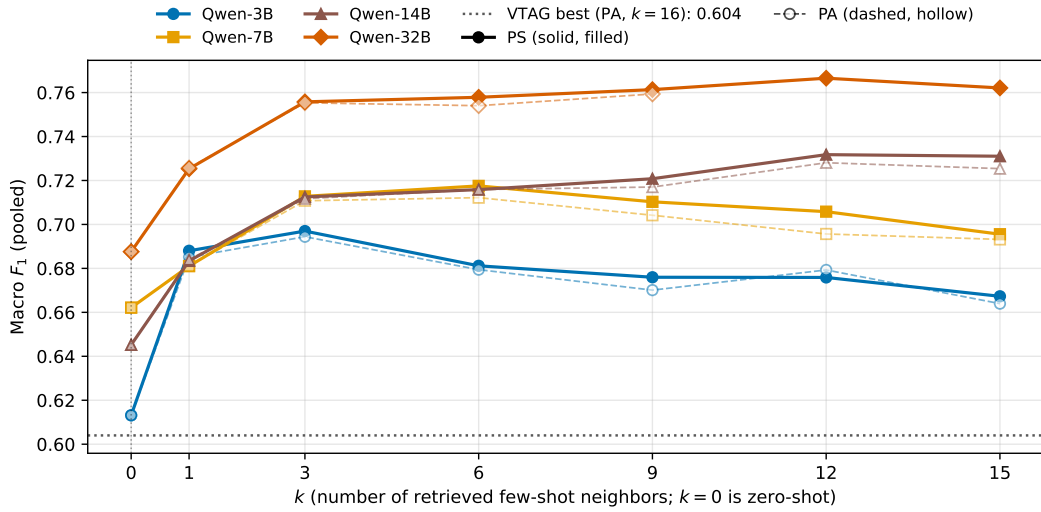
5.2 RAGTAG

We evaluate RAGTAG at k values $\{0, 1, 3, 6, 9, 12, 15\}$ on both PA and PS settings, where $k=0$ is the zero-shot baseline [7]. k value grid selection is informed by VOTAG’s peak results around $k=15$ in the previous section. Figure 2 shows the macro F_1 of every model across this grid for both settings.

PS marginally outperforms PA at almost every (model, k) pair evaluated. The close performance gap is due to the same top- k neighbor overlap observed in the VOTAG analysis. The marginal improvement is significant because PS uses $11\times$ less retrieval data per project: each PS index covers only its own 300 retrieval issues, while PA uses the full 3,300 across all 11 projects. Given this practical advantage, we conduct the rest of our RAGTAG analysis on the PS setting and report PS numbers throughout this section.

The best k grows with model size: Qwen-3B peaks at $k=3$ (macro $F_1 = 0.697$), Qwen-7B at $k=6$ (0.718), and both Qwen-14B and Qwen-32B at $k=12$ (0.732 and 0.767 respectively). This shows that larger models can use the context from additional examples more effectively. However, adding neighbors past $k=12$ has no benefit and slightly degrades performance: every model loses 0.001–0.015 macro F_1 going from $k=12$ to $k=15$.

Every RAGTAG configuration outperforms VOTAG’s best (0.604 at PA $k=16$), with the smallest margin from Qwen-3B zero-shot ($F_1 = 0.613$, +0.009) and the largest from Qwen-



■ **Figure 2** RAGTAG macro F_1 as a function of k for all four Qwen sizes. Solid lines with filled markers are project-specific (PS); dashed lines with hollow markers are project-agnostic (PA). $k=0$ is the zero-shot baseline. The dotted horizontal line marks VOTAG’s best (0.604, PA $k=16$).

32B PS at $k=12$ ($F_1 = 0.767$, $+0.163$). Overall, this shows that adding the LLM’s semantic understanding can meaningfully improve IRC on top of pure retrieval. It is impressive that VOTAG, a cost-efficient retrieval-only heuristic with no LLM reasoning, comes within 0.009 macro F_1 of Qwen-3B zero-shot.

Similar to VOTAG, question consistently has the weakest performance among the three labels across all RAGTAG settings. Figure 3 shows per-class F_1 at each model’s best k . Even the smallest gaps between question and the other labels, observed at Qwen-32B with $k=12$, are noticeable: question is 0.060 macro F_1 below bug and 0.143 below feature. Questions tend to be more frequently misclassified as bugs, similar to our observations for VOTAG. At zero-shot, 46–59% of true-question issues are predicted as bugs while only 5–16% are predicted as features, a 3–11 $\times$ asymmetry. RAGTAG narrows but does not eliminate this asymmetry: at each model’s best k , 26–36% of question issues are still predicted as bugs and only 9–15% as features, a 2–3 $\times$ asymmetry. The bug-bias therefore persists across all four model scales and across both zero-shot and few-shot retrieval. This indicates that the bias is a structural property of both the LLM prior and the retrieval geometry, not something an off-the-shelf RAGTAG configuration can fix.

6 Discussion

7 Threats to Validity

8 Conclusion

9 Data Availability

References

- 1 Giuliano Antoniol, Kamel Ayari, Massimiliano Di Penta, Foutse Khomh, and Yann-Gaël Guéhéneuc. Is it a bug or an enhancement? a text-based approach to classify change requests.

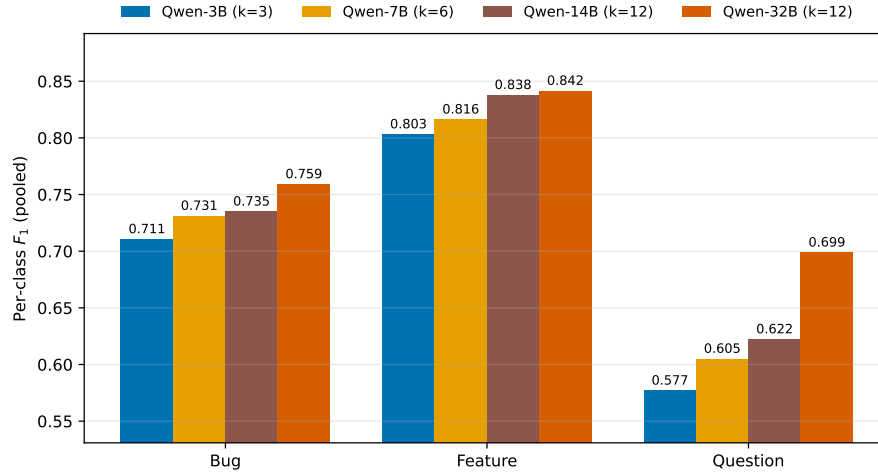


Figure 3 Per-class F_1 for each model size at its best RAGTAG-PS configuration. Question consistently lags both bug and feature across all four models.

- 218 In *Proceedings of the 2008 conference of the center for advanced studies on collaborative*
 219 *research: meeting of minds*, pages 304–318, 2008.
- 220 2 Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A. Gerosa. Applying
 221 large language models api to issue classification problem, 2024. URL: <https://arxiv.org/abs/2401.04637>, arXiv:2401.04637.
- 222 3 Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A Gerosa. Applying
 223 large language models to issue classification: Revisiting with extended data and new models.
 224 *Science of Computer Programming*, page 103333, 2025.
- 225 4 Shikhar Bharadwaj and Tushar Kadam. Github issue classification using bert-style models. In
 226 *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*,
 227 pages 40–43, 2022.
- 228 5 Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla
 229 Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language
 230 models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901,
 231 2020.
- 232 6 Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Issue report classification using
 233 pre-trained language models. In *Proceedings of the 1st International Workshop on Natural*
 234 *Language-based Software Engineering*, pages 29–32, 2022.
- 235 7 Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Benchmarking large language models
 236 for automated labeling: The case of issue report classification. *Information and Software*
 237 *Technology*, page 107758, 2025.
- 238 8 Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. Leveraging gpt-like
 239 llms to automate issue labeling. In *Proceedings of the 21st International Conference on Mining*
 240 *Software Repositories*, pages 469–480, 2024.
- 241 9 Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on*
 242 *information theory*, 13(1):21–27, 1967.
- 243 10 Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-
 244 Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE*
 245 *Transactions on Big Data*, 2025.
- 246 11 Sahibsingh A Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on*
 247 *Systems, Man, and Cybernetics*, (4):325–327, 1976.
- 248

- 249 **12** Qiang Fan, Yue Yu, Gang Yin, Tao Wang, and Huaimin Wang. Where is the road for issue
250 reports classification based on text mining? In *2017 ACM/IEEE International Symposium on*
251 *Empirical Software Engineering and Measurement (ESEM)*, pages 121–130. IEEE, 2017.
- 252 **13** Jueun Heo and Seonah Lee. A study on applying large language models to issue classification.
253 In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, pages
254 1–11. IEEE Computer Society, 2025.
- 255 **14** Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuezhi Li, Shean Wang, Liang
256 Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3,
257 2022.
- 258 **15** Huihui Huang, Ratnadira Widyasari, Ting Zhang, Ivana Clairine Irsan, Jieke Shi, Han Wei
259 Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, et al. Back to the basics:
260 Rethinking issue-commit linking with llm-assisted retrieval. *arXiv preprint arXiv:2507.09199*,
261 2025.
- 262 **16** Maliheh Izadi. Catiss: An intelligent tool for categorizing issues reports using transformers. In
263 *Proceedings of the 1st international workshop on natural language-based software engineering*,
264 pages 44–47, 2022.
- 265 **17** Maliheh Izadi, Kiana Akbari, and Abbas Heydarnoori. Predicting the objective and priority
266 of issue reports in software repositories. *Empirical Software Engineering*, 27(2):50, 2022.
- 267 **18** Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. Ticket tagger:
268 Machine learning driven issue classification. In *2019 IEEE International Conference on*
269 *Software Maintenance and Evolution (ICSME)*, pages 406–409. IEEE, 2019.
- 270 **19** Jiachang Liu, Dinghan Shen, Yizhe Zhang, William B Dolan, Lawrence Carin, and Weizhu
271 Chen. What makes good in-context examples for gpt-3? In *Proceedings of Deep Learning*
272 *Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for*
273 *deep learning architectures*, pages 100–114, 2022.
- 274 **20** Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni,
275 and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions*
276 *of the association for computational linguistics*, 12:157–173, 2024.
- 277 **21** Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy,
278 Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert
279 pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- 280 **22** Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-
281 networks. *arXiv preprint arXiv:1908.10084*, 2019.
- 282 **23** Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’
283 sensitivity to spurious features in prompt design or: How i learned to start worrying about
284 prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.
- 285 **24** Alexander Trautsch and Steffen Herbold. Predicting issue types with sebert. In *Proceedings*
286 *of the 1st international workshop on natural language-based software engineering*, pages 37–39,
287 2022.
- 288 **25** Julian Von der Mosel, Alexander Trautsch, and Steffen Herbold. On the validity of pre-trained
289 transformers for natural language processing in the software engineering domain. *IEEE*
290 *Transactions on Software Engineering*, 49(4):1487–1507, 2022.
- 291 **26** Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan
292 Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv*
293 *preprint arXiv:2109.01652*, 2021.
- 294 **27** An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan
295 Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint*
296 *arXiv:2412.15115*, 2024.